



POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

TESI DI LAUREA MAGISTRALE IN
COMPUTER SCIENCE AND ENGINEERING
INGEGNERIA INFORMATICA

Binary Instrumentation for Tracking Edge Coverage in Fuzzing

Author:

Scandroglio Edoardo

Student ID:

216600

Advisor:

Prof. Name Surname

Academic Year:

2026-27

Dedicated to someone.

Abstract

...

Keywords: key, words, go, here

Abstract in lingua italiana

...

Parole chiave: qui, vanno, le, parole, chiave

Contents

Abstract	i
Abstract in lingua italiana	iii
Contents	v
1 Background and Motivation	1
1.1 Fuzzing Background	1
1.2 Binary Rewriting Background	4
1.3 Problem Statement	7
1.4 State of the Art	8
1.4.1 AFL Functioning	9
1.5 Goals and Challenges	12
Bibliography	15
List of Acronyms	19
List of Figures	21
List of Tables	23
List of Algorithms	25

1 | Background and Motivation

This chapter will focus on presenting the context of this thesis. In particular in Section 1.1 the fundamentals of fuzzing will be discussed with a particular attention on coverage-guided grey-box fuzzing. In Section 1.2 is discussed the background on binary rewriting, its approaches and its difficulties. Section 1.3 states the problem addressed by this work, and Section 1.4 reviews the state of the art in binary-only fuzzing, focusing also on a description of AFL/AFL++ internals. In the last section (Section 1.5) I will present which are the main goals and challenges of this thesis, and why the implemented tool differ from the state of the art .

1.1. Fuzzing Background

Fuzzing is nowadays a widely used approach for software testing and vulnerability assessment [17, 26]. Its main strenght derives from the fact that could rapidly produce various inputs to be fed into a target executable. In general a fuzzer can be divided into the **Input Generator**, that is responsible of provide inputs to the **Executor** that execute them. During the execution the **Defect Monitor** check if the input executed ends in a crash of the target.

In particular the Input Generator must choose which input provide to the executor between all the inputs in the *Input Space*. A subset of the *Input Space* is represented by the *Valid Input Space*. This is a subset of the input space, specifically contains all the inputs that are well formed with respect to the application that is being tested. For example if the fuzzed target if a pdf-reader, a valid input is represented by a pdf file. A good Input Generator must choose inputs that are also valid inputs, because only by doing this there is a probability of find a vulnerability[26]. So the need of a precise input selection and mutation arise, in order to discover as many edge cases as possible.

In order to do so, the Input Generator must gain some informations from the Executor. Depending on the amount of informations that the Input Generator has, fuzzing can be classified into *White-box*, *Black-box* or *Grey-box*. In Black-box fuzzing the fuzzer does not have any knowledge about the execution [19]. In White-box fuzzing instead the source

code must always be available and technique of Symbolic and Concolic execution are used [11]. In Grey-box fuzzing instead the inputs are chosen using, for example, **Code Coverage** informations (i.e. number of lines of code visited during execution of the target). This approach is called *Coverage-Guided Grey-box fuzzing* and is based on prioritizing the inputs that discover new sections of code. Since not always the source code is available the Code Coverage is not always a viable Coverage Measure. In general the coverage measures could be of different granularity [10]. The main three options are:

- **Block Coverage** : Keep track of the hit count for each Basic Block (BB). This approach is used by some popular fuzzers such as libFuzzer or honggfuzz [10]. This method is often too imprecise because it fails to capture the transitions between bb
- **Edge Coverage** : Keep track of the hit count of every edge in the Control Flow Graph (CFG) (i.e. transition between two BB). This is the approach used by AFL and AFL++ [8]. Maintain the overheads similar to Block Coverage but could track the transition between BB and not only their execution adding some context information. Beside this it lack of precision in keep track of the order in which the edges are taken or not.
- **Path Coverage** : This is the most complete metric and keep track of the complete path taken during the execution of a specific input, so keeping track also of the order in which the edges are taken. The main drawback of this metric is represented by the fact that a single execution path could become very large, and so incur in large runtime and memory overheads.

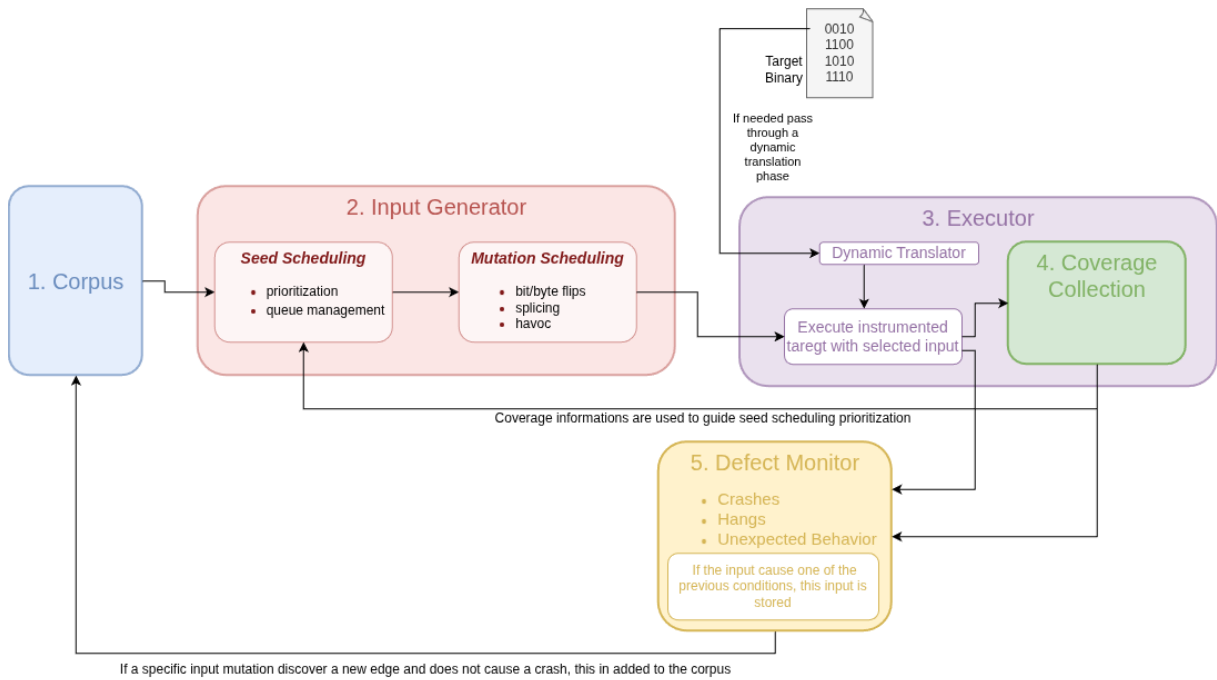
The best approach to Code Coverage, in terms of overhead and effectiveness, is **Edge Coverage** [10] which keeps track of which edges in the CFG are taken during execution but without the overheads given by keeping track of all the execution path. The coverage informations are obtained through the instrumentation of the target application. Depending on the availability of the source code, the instrumentation could be added in different ways:

- The source code is available : In this case the instrumentation is added at *Compile Time*
- The source code is not available : In this case there are two different techniques to instrument the application:
 - **Static Binary Instrumentation (SBI)** : The instrumentation is added before the target is executed but without having the source code, so relying on Static Binary Rewriting techniques.

- **Dynamic Binary Instrumentation (DBI)** : The instrumentation is added during the program execution, relying on Dynamic Binary Rewriting techniques.

Independently from the availability of the source code, *Coverage-Guided Grey-box fuzzing* technique could be summarized as seen in fig. 1.1.

Figure 1.1: Generic diagram that describe Coverage-Guided Grey-box fuzzing workflow



Specifically for **Edge Coverage** the fig. 1.1 could be described as follows:

1. **Seed Corpus** : Represent the set of inputs on which are computed the mutations. At the beginning is composed of user defined seeds but during fuzzing this set can grow if a specific mutation discover new edges not yet discovered.
2. **Input Generator** : Is composed of two main parts:
 - **Seed Scheduling** : responsible of decide which seed in the corpus to mutate
 - **Mutation Scheduling** : responsible of decide which mutation or mutations apply to the selected seed
3. **Executor** : Is the component that actually execute the target binary. If the target binary has already been instrumented with Edge Coverage tracking instruction is executed as is, otherwise, in case of DBI a dynamic translator must dynamically

instrument the binary.

4. **Coverage Collection** : In case of Edge Coverage this part is represented by the data structure used to keep track of which edges between BB has been taken and which are not.

These information are used, in some tools such as AFL++ [8] to guide Seed Scheduling, a technique that, given different inputs with the same level of coverage (i.e. number of edges visited), prioritize the input that is smaller in size and takes less execution time.

5. **Defect Monitor** : this component check for exit status of the target to catch any crash or for an eventual timeout of the target or any other unexpected behavior of this. If none of these conditions are met, which means that the target ends normally, the coverage is checked to see if the current input had discovered a new edge. In case the input had discovered a new edge, that input is added to the corpus.

1.2. Binary Rewriting Background

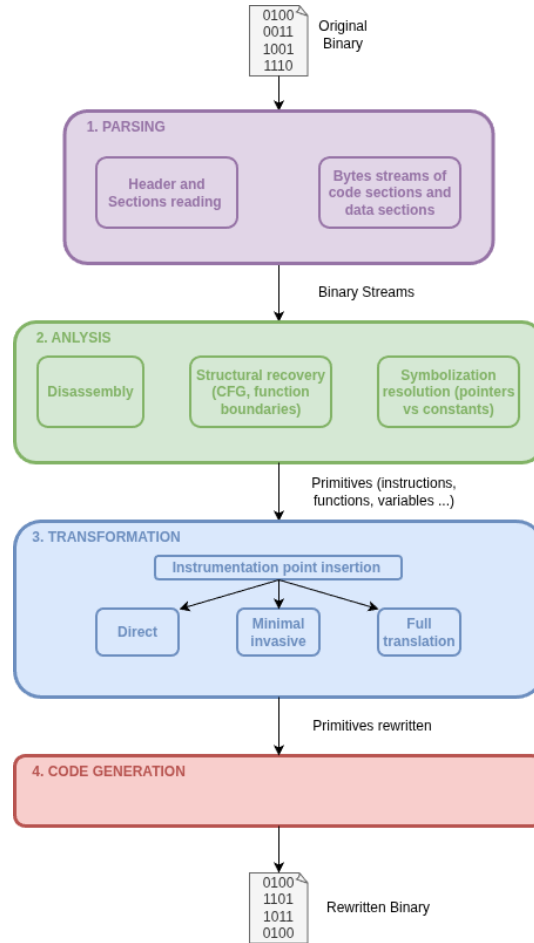
Binary rewriting consist in the alteration af an already compiled executable without having access to the source code while the modified binary remains executable. The process of binary rewriting can be classified into Static Binary Rewriting , if the binary is stored into the persistent memory while rewritten, and Dynamic Binary Rewriting, if the binary is being executed. The process of binary rewriting could be divided into four main steps [22]:

1. *Parsing* : In this phase the main goal is to retrieve informations about the binary. In particular the output of this phase is a raw stream of instruction (in machine code) and data. These informations are then used in the Analysis step
2. *Analysis* : The informations gained from the Parsing phase are analyzed to reconstruct the program structure. For example the raw stream of machine code is passed through a disassembler that reconstruct the assembly instructions. A structural recory is performed, that include function detection and CFG reconstruction, labels and symbols are extracted from the raw stream of data.
3. *Transformation* : Once the CFG has been reconstructed and functions and data boundaries has been recovered, the binary rewriter can modify the binary at the Instrumentation Point (IP). These are user defined locations in the binary where the rewriter must add, modify or delete an instruction. The IP could be coarse or fine-grained. In the first case, for example, the instrumentation could be made at branch

level (at the beginning of each BB). In the second case instead any instruction could be potentially modified.

4. *Code Generation* : In this last step the changes made in the previous steps must be integrated into the binary preserving its executability. This could be done in different ways. For example the changes could be inserted into a new section of the executable and redirect the execution to the new code at the IP. Otherwise this step could be done also reassembling a completely new executable starting from the changes made at IP. This second approach is much harder because need to preserve all the references inside the excutable and preserve the executable layout.

Figure 1.2: An example of Static Binary Rewriting Workflow

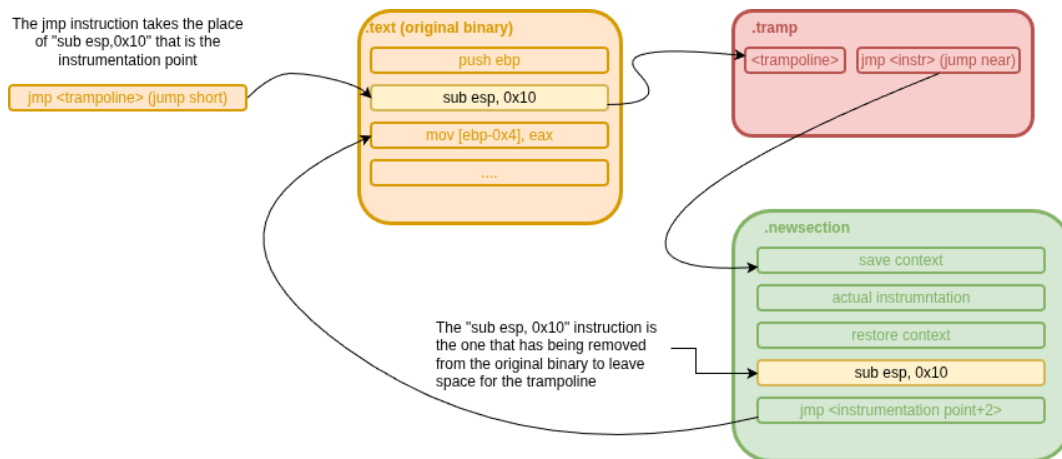


In fig. 1.2 can be seen a general scheme for static binary rewriting. In this case all the phases occur prior the execution. So the cost in terms of overheads is paid only one time before the execution. The main problems are disambiguate between code and data (i.e. decide if a stream of bytes represent machine code instructions or are constant placed inline into the .text section) and resolving indirected branches [3, 22]. This problem is

in general undecidable and can be resolved only using heuristics. There are three main families of static transformation:

- **Direct** : Consist in directly modifying the instruction. Works well with RISC systems with instructions having the same length. Generally fails to add and remove instruction because this doesn't preserve the offset into the .text section.
- **Minimal-invasive**: The new code is inserted into a new section of the binary and in the IP is placed an unconditional jump to the new section. The main limitation of this approach arise when the new section is far from the IP and a jump near (necessary to reach the new section) uses a number of bytes different from the number of bytes used by the instruction at the instrumentation point. Infact in assembly x86-64 a jump short uses 2 bytes and a jump near uses up to 5 bytes. So in case at the instrumentation point is needed a jump short for alignment reason, but the new section is too far away and a jump near is needed, it is necessary to add a **trampoline**. This approach can be seen in fig. 1.3. The trampoline based instrumentation is one of the most common approach to binary rewriting [22].
- **Full-translation** : The binary is lifted into an Intermediate Representation (IR) that makes possible to add an IP at instruction level. The IR is then reassembled into machine code after the changes.

Figure 1.3: An example of Static Instrumentation with trampoline technique



With regard to Dynamic Binary Rewriting the Analysis and Transformation phases are executed at runtime for each instruction. In this case there are not all the problems that arise in Static Binary Rewriting, since the Binary is being executed, and so the indirected branches could be resolved with the values inside the registers and there is no need for disambiguate between wich are constants inside the .text section and which are instructions. The main drawback of this approach consist in prohibitive overheads caused by

the dynamic analysis and transformation of each instruction, alongside the fact that only the taken branches could be modified. Infact if a branch is not actually taken during execution, its instructions never reach the rewriter.

A specific type of binary rewriting is the Binary Instrumentation. The binary instrumentation uses binary rewriting techniques to add instructions that tracks binary behaviour, log event or, in the case of this thesis, insert edge coverage tracing instructions.

1.3. Problem Statement

Coverage-Guided Grey-box fuzzing is one of the most effective techniques in automatic vulnerability disclosure on software [6, 20], due to its capability of generating wide amount of inputs based on the feedback obtained through Code Coverage informations that guide the fuzzer towards unexplored regions of code and therefore find hidden vulnerabilities. Infact *Black-box* fuzzing lack of a precise mutation strategy based on execution monitoring while *White-box* fuzzing incur in high overheads given by Symbolic or Concolic execution. Furthermore, the latter approach also requires the availability of the source code, not always available. So *Grey-box* fuzzing seems to be the most promising trade-off between precision of input mutation and overheads. However, also *Grey-box* fuzzing has its own drawback. As matter of facts the effectiveness of this approach strongly depends on the availability of the source code. Infact both SBI and DBI have many limitations. The main limitation of DBI is represented by huge amount of overheads (higher than 600% overheads in case of AFL++ in QEMU-Mode [20]) that makes the exploration of a substantial part of the valid input space almost impossible. Instead SBI main difficulty arise in modifying already compiled binaries. There are several problems regarding Binary Rewriting such as Indirect Control Transfers (e.g. `call rax`), shift in binary sections or instructions that cause the need of rewrite the instructions that reference these sections and many others as seen in the previous Section 1.2.

For all these reasons the preferable technique nowadays is *Compile time Edge Coverage instrumentation* with fuzzers such as AFL++, LibAFL [9] o libFuzzer [15]. However in practice, for industrial secrecy (i.e. proprietary software) or due to precompiled libraries, the applications are often closed source, and so Compile Time Edge Coverage instrumentation become unusable.

In these scenario binary only fuzzing is the only viable approach. Although SBI presents many challenges, it nonetheless proves to be the most fruitful area, given the low overheads [20]. In this thesis I will address the problem of statically instrument a binary with Edge Coverage instructions in a way that the instrumented binary is compatible with

AFL++, which has been chosen as fuzzer due to its popularity and because represent the current state of the art implementing most of the recent discovered techniques disclosed demonstrated by Andrea Fioraldi et al. [8].

The instrumentation must add as little overheads as possible with respect to compile time overheads and must preserve the correctness of the binary. This is important in fuzzing because less overheads means more executions per second, so a bigger portion of the valid input space could be visited.

1.4. State of the Art

In this section I will disclose some of the current state of the art solutions in terms of binary only greybox fuzzing, differentiating between SBI and DBI approaches.

- **DBI**

- **AFL++ in QEMU/FRIDA mode**[8] : AFL++ in QEMU mode uses QEMU in *User-mode Emulation* which consist in translating guest machine code (i.e. guest BB) into an intermediate representation called Translation Block (TB) and subsequently append the coverage tracing instructions at the end of it. This approach is based on emulating the entire CPU thanks to QEMU virtualized environment. As stated before, AFL in QEMU mode has overheads in the order of 600% with respect to the compile time instrumentation [20]

Instead AFL++ in FRIDA mode [1] is based on a shared library injection in which is present a frida [21] component called **Stalker** which intercept any BB that is about to be executed , copy it into a **Code Cache** and dynamically instrument the copy with coverage tracing instructions. The execution control is then transferred to the copy, execute it, and then re-intercepted by **Stalker** at the end. Differently from QEMU mode doesn't emulate the CPU, but is executed within the memory space of the target program. AFL++ reports that FRIDA mode is approximately 2 to 5 times slower than compile-time instrumentation [2].

- **DynamoRIO/Pin**[4, 16] : Both these approaches share the same base principle, called **Just in-time (JIT) code caching**. This consist on take control of the thread when the target application starts, and from that moment on every BB is intercepted before it is executed and is copied into a private **Code Cache**. The instrumentation is then added to the copy and the control is transferred to the code cache. The instrumentation process is more or less identical to

FRIDA’s Stalker mechanism, but are not optimized like taht. Also in this case the implemented version uses AFL[25] as fuzzer [13, 14].

- **SBI**

- **Retrofit** [7]: This tool performs symbolic lifting on the code, replacing every reference discovered during disassembly with a symbolic label. Doing this allows instructions to be added without modifying the references, achieving reassembleable assembly. While RetroWrite can add ASan instrumentation, it cannot natively add AFL-style edge coverage instructions. To do so, the lifted assembly must be instrumented with the afl-gcc tool in order to produce the patched binary. Moreover, community-reported issues [18] document missing instrumentation points when RetroWrite’s symbolized output is compiled through afl-gcc, caused by a mismatch between RetroWrite’s label-naming convention and afl-as’s basic-block detection heuristics. Finally, RetroWrite only works with PIE, non-stripped binaries.
- **AFL-Dyninst** [12] : AFL-Dyninst employs Dyninst’s binary rewriting capabilities to instrument an existing executable by inserting callbacks at selected basic blocks. Dyninst’s instrumentation mechanism is based on code patching and trampoline redirection [5], which cause many context switches and state saving/restoring. Its performance are slightly better than DBI but this approach reaches 500% of fuzzing overheads as demonstrated by Stefan Nagy et al. [20].
- **ZAFL** [20] : This tool uses two main phases: the first phase consist in reconstructing an IR using the Zipr binary rewriter. The obtained IR is then used by ZAX to perform an optimization on the IR, find the instrumentation points and in the end instrument these points. ZAFL works inserting calls to a function that update the shared map at the beginning of each BB. Beside many improvements of ZAFL, Jifan Xiao et al. [23] demonstrated that still cause crashes in binaries in 28% of the binaries used in the cited paper.

Before proceeding with the next section in which I will talk about the main goals of this thesis, I want to discuss the core functioning of AFL and AFL++, given its importance for understanding my approach.

1.4.1. AFL Functioning

[24] As previously stated, AFL and, in particular, AFL++, are among the most widely

used grey-box fuzzing frameworks [8]. AFL++ represent the community based continuation of the initial AFL project. The core functioning is virtually the same and consist in a Shared Memory (SHM) region, shared between the fuzzer and the target. The SHM dimension could vary but the default one is of 65536 bytes.

Each “cell” of the SHM contains the number of times a given Edge in the CFG has been taken during the execution of the target. Since the SHM has 65536 “cells”, each “cell” can contain a number from 0 to 255. The “cell” position is computed with a specific hash function applied to the IDs of the two BB that formed the edge. The main Edge Tracing update rule is summarized in algorithm 1.1. The line line 3 of algorithm 1.1 is used to

Algorithm 1.1 AFL Style SHM update rule

```

1:  $idx \leftarrow current\_bb\_id \oplus previous\_bb\_id$ 
2:  $shared\_map[idx] \leftarrow shared\_map[idx] + 1$ 
3:  $previous\_bb\_id \leftarrow current\_bb\_id \gg 1$ 

```

avoid update the same SHM location if an edge is traversed forward or backward. An example of this could be a simple **do-while** loop as seen in algorithm 1.2 and in fig. 1.4.

The main idea behind AFL is instrumenting each basic block of a target application

Algorithm 1.2 Do-While Example

```

repeat
  {Basic Block A }
until Permanence_Condition
  {Basic Block C}

```

Without line 3 of algorithm 1.1 the hash of Edge1 will result in $ID_A \oplus ID_B$ and the hash of Edge2 (in case the permanence condition is not met) will result in $ID_B \oplus ID_A = ID_A \oplus ID_B$ for xor simmetry. But since the hash is the same there will be an hash collision that cause the same “cell” to be updated two times instead on one.

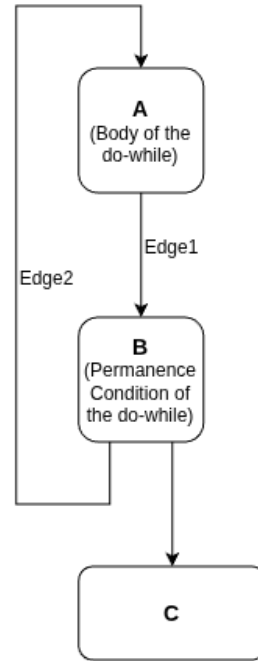


Figure 1.4: Do-while CFG

with the shared map update rule. The fuzzer initialize the SHM memory with zeros, run the instrumented target that will update the SHM with the algorithm 1.1, until it reach the end and then return to the fuzzer. The SHM ID is comunicated to the target with an

environmental variable called `__AFL_SHM_ID`.

At this point the fuzzer check the just obtained SHM, that il called `trace_bits`, searching for any new discovered edge. This is done keeping a structure called `virgin_bits` that contains all the edges not yet discovered and it is initialized with ones. The compare is executed through `trace_bits & virgin_bits`. If one of the position gives one after the logic AND, this means a new edge has been discovered. In this case the `virgin_bits` data structure is updated and the just executed input is saved in the corpus. Otherwise a new input is choosen, mutated and then executed.

This approach requires that for each input a `fork()` call and then an `execve()` call are performed, causing a lot of overheads for each input execution. Two main optimizations are been implemented:

- **forkserver**
- **persisten-mode** : (vailable only in AFL++)

Forkserver

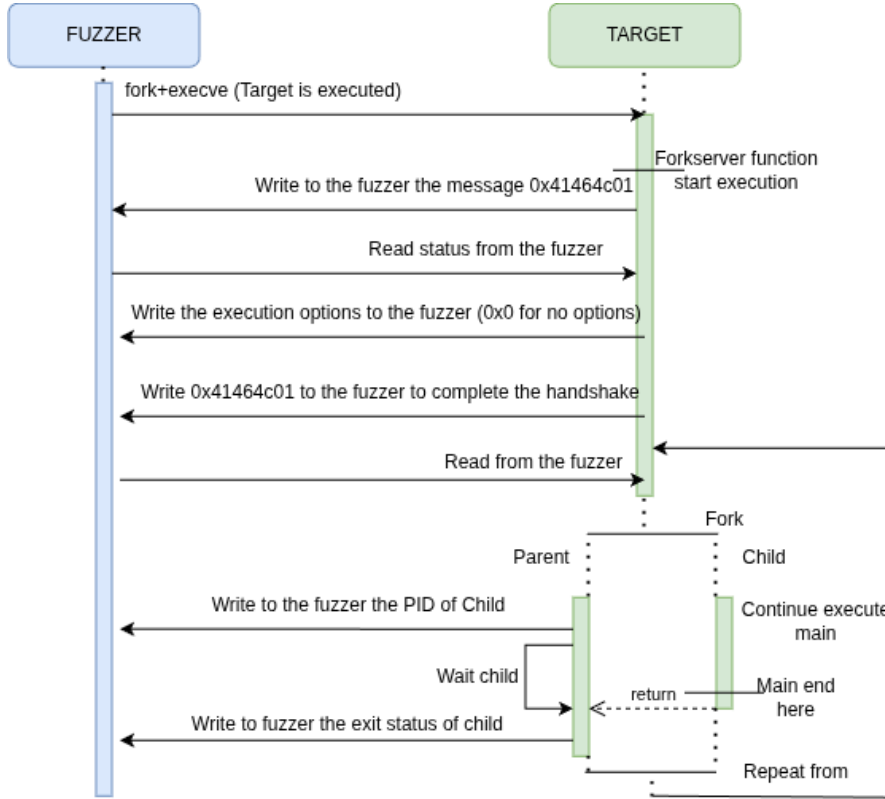
The forkserver makes it possible to avoid calling `execve` for every input, performing it only once at the start of execution. The forkserver is instrumented before the main of the target application. The AFL++ forkserver protocol could be summarized as follows: ¹ The communication between Fuzzer and Target seen in fig. 1.5 happens unidirectionally through two different file descriptors created by the fuzzer:

- *`FORKSRV_FD` (fd 198)*: used by the forkserver to read a message sent by the fuzzer.
- *`FORKSRV_FD + 1` (fd 199)*: used by the forkserver to write a message to the fuzzer.

1. The forkserver sends the “Welcome” message to the fuzzer (0x41464c01).
2. The fuzzer reply with the status code. At this point the forkserver must check if `status & 0xffffffff = 0x41464c01`. If it is equal continue the execution, otherwise the forkserver stops here.
3. The forkserver sends the execution options to the fuzzer. These could indicate the fact that the target is running in persisten mode or other optimizations.
4. The forkserver sends 0x41464c01 to the fuzzer to end the handshake

¹ *The differences between AFL forkserver protocol and AFL++ forkserver protocol are mainly the messages exchanged between forkserver and fuzzer*

Figure 1.5: AFL++ Forkserver Protocol



5. The fuzzer write a message to the forkserver (its content is discarded) the handshake is finished. This message is the “go” signal for the forkserver.
6. The forkserver invoke `fork()`. The parent write the PID of the child to the fuzzer and then invoke `waitpid()`.
The child continue the execution of the main until it ends
7. When the parent exit from the `waitpid()` function (i.e. the child terminate), the exit status of the child is sent to the fuzzer and the forkserver waits for another “go” from the fuzzer.

1.5. Goals and Challenges

The main goal of this thesis is to develop a tool capable of inline, static edge-coverage instrumentation that preserves binary correctness while achieving performance comparable to compile-time instrumentation.

I have choose to implement the SBI approach instead of DBI due to its low overheads. For doing this I will use GRAPE binary rewrite framework, a novel approach for binary rewriting that preserve binary executability, while introducing small overheads with re-

spect to a trampoline based approach. This approach involves several challenges, mainly related to preserving the binary's correctness and behavior. Moreover the tool must be compatible with the already existing AFL++ fuzzer. This means implement the fork-server communication protocol inside the target binary without recompiling it. This had created not few problems, mainly regarding the preservation of ELF layout after adding new sections. With respect to ZAFL approach, the SHM update instructions are placed directly inside the BB, and not in a function called at each BB. This approach increment drastically the dimension of the file, causing many instruction repetitions (infact the SHM instructions are always the same), but has been chosen because does not cause a context switch at each BB. Infact, in order to call a function, the state of the registers must be saved and restored. Moreover stack must be allocated, the callee RIP must be saved on the stack alongside the old Base Pointer of the stack. All these operations to be performed at each BB introduce significant overheads.

Bibliography

- [1] AFL++ contributors. AFL++ Frida Mode. https://github.com/AFLplusplus/AFLplusplus/tree/stable/frida_mode, 2026. Accessed: 2026-09-13.
- [2] AFL++ Developers. Fuzzing binary-only targets. URL https://aflplusplus/docs/fuzzing_binary-only_targets/.
- [3] D. Andriesse, X. Chen, V. van der Veen, A. Slowinska, and H. Bos. An in-depth analysis of disassembly on full-scale x86/x64 binaries. In *25th USENIX Security Symposium (USENIX Security 16)*, pages 583–600, Austin, TX, 2016. USENIX Association. ISBN 978-1-931971-32-4. URL <https://www.usenix.org/conference/usenixsecurity16/technical-sessions/presentation/andriesse>.
- [4] D. Bruening, T. Garnett, and S. Amarasinghe. An infrastructure for adaptive dynamic optimization. In *International Symposium on Code Generation and Optimization, 2003. CGO 2003.*, pages 265–275, 2003. doi: 10.1109/CGO.2003.1191551.
- [5] B. Buck and J. K. Hollingsworth. An api for runtime code patching. *The International Journal of High Performance Computing Applications*, 14(4):317–329, 2000. doi: 10.1177/109434200001400404. URL <https://doi.org/10.1177/109434200001400404>.
- [6] M. Böhme, V.-T. Pham, and A. Roychoudhury. Coverage-based greybox fuzzing as markov chain. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, CCS16, pages 1032–1043. ACM, Oct. 2016. doi: 10.1145/2976749.2978428.
- [7] S. Dinesh, N. Burow, D. Xu, and M. Payer. Retrowrite: Statically instrumenting cots binaries for fuzzing and sanitization. In *2020 IEEE Symposium on Security and Privacy (SP)*, pages 1497–1511, 2020. doi: 10.1109/SP40000.2020.00009.
- [8] A. Fioraldi, D. Maier, H. Eißfeldt, and M. Heuse. AFL++ : Combining incremental steps of fuzzing research. In *14th USENIX Workshop on Offensive Technologies (WOOT 20)*. USENIX Association, Aug. 2020. URL <https://www.usenix.org/conference/woot20/presentation/fioraldi>.

- [9] A. Fioraldi, D. C. Maier, D. Zhang, and D. Balzarotti. Libafl: A framework to build modular and reusable fuzzers. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, pages 1051–1065, 2022.
- [10] S. Gan, C. Zhang, X. Qin, X. Tu, K. Li, Z. Pei, and Z. Chen. Collafl: Path sensitive fuzzing. In *2018 IEEE Symposium on Security and Privacy (SP)*, pages 679–696, 2018. doi: 10.1109/SP.2018.00040.
- [11] P. Godefroid, M. Y. Levin, and D. Molnar. Sage: whitebox fuzzing for security testing. *Communications of the ACM*, 55(3):40–44, Mar. 2012. ISSN 1557-7317. doi: 10.1145/2093548.2093564.
- [12] M. Heuse. Afl-dyninst, 2018. URL: <https://github.com/vanhauser-thc/afl-dyninst>.
- [13] M. Heuse. AFL-DynamoRIO. <https://github.com/vanhauser-thc/afl-dynamorio>, 2018. Accessed: 2026-09-13.
- [14] M. Heuse. AFL-PIN. <https://github.com/vanhauser-thc/afl-pin>, 2018. Accessed: 2026-09-13.
- [15] LLVM Project. libfuzzer – a library for coverage-guided fuzz testing. <https://llvm.org/docs/LibFuzzer.html>. Accessed: 2026-09-11.
- [16] C.-K. Luk, R. Cohn, R. Muth, H. Patil, A. Klauser, G. Lowney, S. Wallace, V. J. Reddi, and K. Hazelwood. Pin: building customized program analysis tools with dynamic instrumentation. *SIGPLAN Not.*, 40(6):190–200, June 2005. ISSN 0362-1340. doi: 10.1145/1064978.1065034. URL <https://doi.org/10.1145/1064978.1065034>.
- [17] S. Mallisery and Y.-S. Wu. Demystify the fuzzing methods: A comprehensive survey. *ACM Computing Surveys*, 56(3):1–38, Oct. 2023. ISSN 1557-7341. doi: 10.1145/3623375.
- [18] Marsman1996. Missing some instrumentations when instrumenting binary with AFL. <https://github.com/HexHive/retrowrite/issues/28>, Aug. 2021. GitHub Issue #28, HexHive/retrowrite repository. Accessed: 2026-09-13.
- [19] B. P. Miller, L. Fredriksen, and B. So. An empirical study of the reliability of unix utilities. *Communications of the ACM*, 33(12):32–44, Dec. 1990. ISSN 1557-7317. doi: 10.1145/96267.96279.
- [20] S. Nagy, A. Nguyen-Tuong, J. D. Hiser, J. W. Davidson, and M. Hicks. Breaking through binaries: Compiler-quality instrumentation for better binary-only fuzzing.

- In *30th USENIX Security Symposium (USENIX Security 21)*, pages 1683–1700. USENIX Association, Aug. 2021. ISBN 978-1-939133-24-3. URL <https://www.usenix.org/conference/usenixsecurity21/presentation/nagy>.
- [21] O. A. V. Ravnås. Frida: Dynamic instrumentation toolkit for developers, reverse-engineers, and security researchers, 2026. URL <https://github.com/frida/frida>.
- [22] M. Wenzl, G. Merzdovnik, J. Ullrich, and E. Weippl. From hack to elaborate technique—a survey on binary rewriting. 52(3), 2019. ISSN 0360-0300. doi: 10.1145/3316415. URL <https://doi.org/10.1145/3316415>.
- [23] J. Xiao, P. Jiang, Z. Zhao, R. Huang, J. Liu, and D. Li. Robust, efficient, and widely available greybox fuzzing for COTS binaries with system call pattern feedback. In *34th USENIX Security Symposium (USENIX Security 25)*, pages 6239–6258, Seattle, WA, Aug. 2025. USENIX Association. ISBN 978-1-939133-52-6. URL <https://www.usenix.org/conference/usenixsecurity25/presentation/xiao-jifan>.
- [24] M. Zalewski. Technical "whitepaper" for afl-fuzz. https://github.com/google/AFL/blob/master/docs/technical_details.txt. Accessed: 2026-09-19.
- [25] M. Zalewski. American Fuzzy Lop. <http://lcamtuf.coredump.cx/afl/>, 2017. Accessed: 2026-09-13.
- [26] X. Zhu, S. Wen, S. Camtepe, and Y. Xiang. Fuzzing: A survey for roadmap. *ACM Computing Surveys*, 54(11s):1–36, Jan. 2022. ISSN 1557-7341. doi: 10.1145/3512345.

List of Acronyms

BB Basic Block. 2, 4, 5, 8–10, 13

CFG Control Flow Graph. 2, 4, 10

DBI Dynamic Binary Instrumentation. 3, 7–9, 12

IP Instrumentation Point. 4–6

IR Intermediate Representation. 6, 9

JIT Just in-time. 8

SBI Static Binary Instrumentation. 2, 7–9, 12

SHM Shared Memory. 10, 11, 13

TB Translation Block. 8

List of Figures

1.1	Generic diagram that describe Coverage-Guided Grey-box fuzzing workflow	3
1.2	An example of Static Binary Rewriting Workflow	5
1.3	An example of Static Instrumentation with trampoline technique	6
1.4	Do-while CFG	10
1.5	AFL++ Forkserver Protocol	12

List of Tables

List of Algorithms

1.1	AFL Style SHM update rule	10
1.2	Do-While Example	10

